ARCGIS AUTOMATION & SPATIAL ENGINEERING

Parcel Division System Technical Specification

An in-depth architectural breakdown, code analysis, and mathematical framework for automated polygon parcel subdivision using ArcPy, polar coordinate projection, and dynamic step convergence.

Author: GIS Software Architecture Group **Environment:** ArcGIS Pro / ArcPy **Language:** Python 3.x

1. Executive Summary & System Purpose

In cadastre management and land surveying, splitting a parcel into sub-lots of exact target areas along precise directional vectors (e.g., perpendicular to a road frontage or aligned along custom bearing angles) is a recurring challenge. Manual editing tools in GIS desktop suites are iterative, time-intensive, and prone to precision errors.

The custom ArcPy solution documented here automates this process. It parses parcel boundaries, executes polar coordinate geometry calculations to construct slicing polylines, dynamically adjusts cutting offsets until area target thresholds are met within strict numerical precision tolerances, and manages schema propagation across the resulting sub-features.

Precise Area Allocation

Divides parent polygons into precise sub-lots based on explicit square-meter allocations (A_{target}) while adhering to user-defined precision thresholds (ϵ).

Arbitrary Angle Support

Supports orthogonal slicing ($0^\circ, 90^\circ$) and arbitrary angled cuts ($\theta \in (0^\circ, 180^\circ)$) measured relative to geographic grid orientation.

Schema & Metadata Integrity

Maintains dynamic attribute mapping, preserving original feature field definitions, field types, and historical attribute records across generated sub-lots.

Adaptive Step Convergence

Employs an adaptive decade step-down algorithm that scales increment sizes ($10 \times \epsilon \rightarrow \epsilon$) to minimize geometry intersection iterations.

2. System Architecture & Input Parsing

The script accepts four primary inputs from the ArcGIS Geoprocessing environment:

- **Input Polygon Layer (`strat`):** Feature class containing parent parcels to be subdivided.
- **Split Angle (`unghi`):** Directional bearing θ in degrees ($0 \le \theta < 180$).
- **Owner & Area Specification (`text`):** Formatted string pairing owner names with area targets (e.g., `"Nicoara 8000;Paul 2000;X-ulescu 5000"`).
- **Area Precision (`precizie`):** Floating-point tolerance threshold ϵ (e.g., 0.01 m^2).

The listing string is parsed into synchronized arrays using string tokenization: target owner names are isolated into an array, while target areas are cast to floating-point numbers. Summed area validation ensures that total sub-lot allocations do not exceed parent parcel geometry area.

3. Mathematics & Coordinate Geometry

Cutting polylines are constructed dynamically across the polygon extent using polar coordinate transformations. Given an initial reference point (X_0, Y_0), a cutting vector end point (X_1, Y_1) is derived over a extended diagonal distance d (calculated as $2 \times \text{envelope diagonal}$) to guarantee full polygon boundary intersection.

> **Polar Coordinate Endpoint Transformation**
> To align standard Cartesian angles with geographic azimuth bearings, the input angle θ is transformed to radians:
>
> $$\theta_{rad} = \frac{\pi}{2} - \text{radians}(\theta)$$
>
> $$X_1 = X_0 + d \cdot \cos(\theta_{rad}) \quad | \quad Y_1 = Y_0 + d \cdot \sin(\theta_{rad})$$

The helper function `point_pos(x, y, d, unghi)` implements this transformation, returning an `arcpy.Point` object that forms an extended `arcpy.Polyline` geometry passing through the target spatial domain.

4. Algorithmic Slicing & Convergence Mechanics

Subdivision relies on an iterative geometry cutting loop using ArcPy's `polygon.cut(polyline)` routine. The algorithm branches into specialized cutting logic depending on the input angle θ:

Angle Branch (θ)	Origin Point Reference	Search Vector & Offset Mechanism
$\theta = 0^\circ$ (Vertical N-S)	Envelope Bounding Box Midpoint	Binary interval division ($\Delta X / 2$) shifting cutting line east/west along X-axis.
$\theta = 90^\circ$ (Horizontal E-W)	Envelope Bounding Box Midpoint	Iterative step convergence shifting baseline north/south along Y-axis.
$0^\circ < \theta < 90^\circ$ (NE-SW)	Bottom-Left Envelope Corner $(X_{\min}, Y_{\min})$	Advances origin offset along X-axis using dynamic step size `valoare_modificare`.
$90^\circ < \theta < 180^\circ$ (NW-SE)	Top-Left Envelope Corner $(X_{\min}, Y_{\max})$	Advances origin offset along X-axis, projecting cutting vector southeastward.

Adaptive Decade Step-Down Convergence

To achieve rapid spatial convergence without excessive compute overhead, the step increment size `valoare_modificare` is dynamically adjusted. Initial step size begins at $10 \times \epsilon$. As the cutting line steps across the feature extent, the cut area A_{cut} is evaluated against target area A_{target}:

- If $A_{\text{cut}} < A_{\text{target}}$, the origin shifts forward by `valoare_modificare`.
- When A_{cut} overshoots A_{target}, the step direction reverses and `valoare_modificare` is divided by 10.
- Looping terminates when $|A_{\text{cut}} - A_{\text{target}}| \le \epsilon$, committing the sub-polygon geometry.

5. Code Structure & Implementation Analysis

The core Python/ArcPy codebase utilizes data access cursors (`arcpy.da.SearchCursor` and `arcpy.da.InsertCursor`) to manage high-performance feature creation and dynamic attribute assignment. Below is the commented implementation detailing the helper routines and cutting loops:

```
      import arcpy, math, os from collections import OrderedDict # Helper: Calculate polar point
destination def point_pos(x, y, d, unghi): rad = math.radians(unghi) # Convert bearing angle to
Cartesian standard angle ang = (math.pi / 2.0) - rad return arcpy.Point(x + d * math.cos(ang), y +
d * math.sin(ang)) # Helper: Extract attribute values maintaining schema order def
get_object_values(row, fields): return [row.getValue(f.name) for f in fields if f.type not in
["Geometry", "OID"]] # Main execution entry strat = arcpy.GetParameterAsText(0) # Input parcel
layer unghi = float(arcpy.GetParameterAsText(1)) # Split angle (degrees) text =
arcpy.GetParameterAsText(2) # "Owner1 Area1;Owner2 Area2" precizie =
float(arcpy.GetParameterAsText(3)) # Area tolerance threshold # Parse input owner allocations
proprietari, suprafete = [], [] for pereche in text.split(";"): p, s = pereche.strip().split(" ")
proprietari.append(p) suprafete.append(float(s)) # Retrieve dataset spatial reference & envelope
extent desc = arcpy.Describe(strat) sr = desc.spatialReference extent = desc.extent diag =
math.hypot(extent.width, extent.height) * 2.0 # Extended line distance
```

6. Schema Preservation & Residual Handling

A critical requirement of cadastral software is preserving existing table schemas. The script dynamically extracts non-system field definitions using `arcpy.ListFields()` and constructs schema-matching parameter lists for feature creation.

When a sub-lot is sliced successfully, its geometry is written using an `InsertCursor` alongside its allocated owner name and legacy attributes. The remaining geometry slice (`remaining_polyg`) is re-assigned as the parent shape for the next iteration loop. Once all target area cuts are completed, the final remaining geometry is automatically assigned to the last owner in the allocation list, ensuring 100% geometric conservation without gaps or overlaps.